

1. Propósito del Análisis

Esta sección tiene como finalidad demostrar cómo el CRUD de productos del sistema PaperBoost puede analizarse de forma progresiva mediante modelos conectados: casos de uso, clases de análisis y diagramas de secuencia. El énfasis técnico no reside en la programación de la interfaz, sino en comprender la responsabilidad de cada componente (frontera, control, entidad y repositorio) y asegurar que se conserve la trazabilidad total desde el requisito funcional hasta la interacción dinámica de los objetos

4. Caso base: CRUD de Productos

El sistema debe permitir gestionar la mercancía de la papelería de manera simplificada, facilitando el control de inventario en tiempo real para pequeños emprendimientos. Según el documento SRS de PaperBoost, cada producto registrado en el inventario contará con la siguiente información mínima requerida para su administración comercial:

Dato	Tipo sugerido	Descripción	Validación mínima
SKU	Entero o texto corto	Identificador único o código interno del producto.	Obligatorio, no vacío y no repetido.
Nombre	Texto	Nombre descriptivo del artículo registrado.	Obligatorio; no debe contener solo espacios.
Precio	Double	Precio unitario de venta al público.	Obligatorio y mayor que 0.
Stock	Entero	Cantidad de unidades disponibles en inventario.	Obligatorio; debe ser un número positivo.

5. Requisitos funcionales de nivel 0 para el CRUD

A continuación se listan los requisitos funcionales de nivel 0 correspondientes a las operaciones CRUD del sistema PaperBoost.

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-07	El sistema debe permitir registrar un nuevo producto ingresando manualmente sus datos (Nombre, SKU, Precio, Stock, etc.)	Administrador	Registrar Producto Manualmente (CUC-04)
RF-13	El sistema debe permitir la edición o modificación de los datos de un producto previamente registrado en el catálogo.	Administrador	Modificar datos de producto (CUC-07)
RF-23	El sistema debe permitir eliminar o dar de baja un producto del inventario para que ya no esté disponible.	Administrador	Eliminar / Dar de baja producto (CUC-12)
RF-11	El sistema debe permitir consultar el inventario de productos mostrando la lista completa.	Administrador	Consultar Inventario de Productos (CUC-06)

RF-08	El sistema debe validar la unicidad de SKU y los campos obligatorios antes de guardar o actualizar cambios.	Sistema	Validar campos y unicidad de SKU
-------	---	---------	----------------------------------

7. Código PlantUML: diagrama de casos de uso CRUD

Codigo

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Administrador" as Admin
actor "Sistema" as Sistema

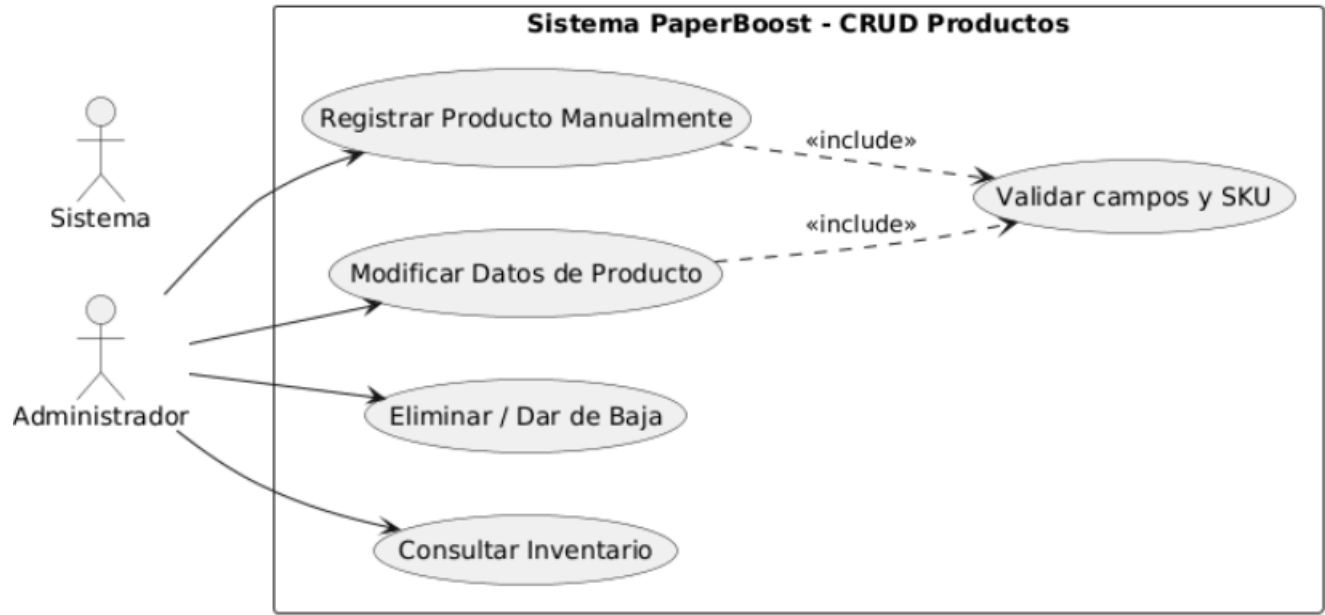
rectangle "Sistema PaperBoost - CRUD Productos" {
    usecase "Registrar Producto Manualmente" as UC1
    usecase "Modificar Datos de Producto" as UC2
    usecase "Eliminar / Dar de Baja" as UC3
    usecase "Consultar Inventario" as UC4
    usecase "Validar campos y SKU" as UC5
}

Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4

UC1 ..> UC5 : <<include>>
UC2 ..> UC5 : <<include>>

@enduml
```

Resultado



PUENTE

Código:

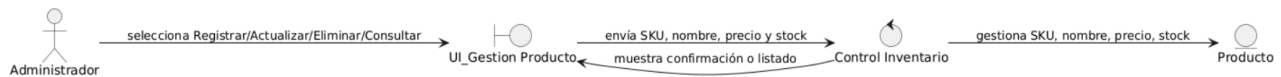
```

@startuml
left to right direction

actor "Administrador" as Admin
boundary "UI_Gestion Producto" as UI
control "Control Inventario" as Ctrl
entity "Producto" as Prod

Admin --> UI : selecciona Registrar/Actualizar/Eliminar/Consultar
UI --> Ctrl : envía SKU, nombre, precio y stock
Ctrl --> Prod : gestiona SKU, nombre, precio, stock
Ctrl --> UI : muestra confirmación o listado
@enduml
  
```

Resultado



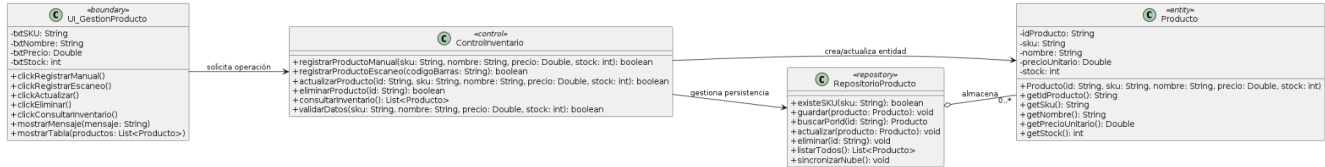
8. Matriz de trazabilidad

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia
RF-07	Registrar Producto Manualmente	Administrador	Alta	El sistema guarda un producto en el catálogo verificando SKU, Nombre, Precio y Stock.	UC1
RF-13	Modificar datos de producto	Administrador	Alta	El sistema modifica los datos del producto elegido luego de validar los campos.	UC2
RF-23	Eliminar / Dar de baja producto	Administrador	Alta	El sistema marca como inactivo y remueve al producto del catálogo activo.	UC3
RF-11	Consultar Inventario de Productos	Administrador	Alta	El sistema muestra una tabla o lista detallada con los registros actualizados.	UC4
RF-08	Validar campos y SKU	Sistema	Alta	El sistema impide guardar registros con campos vacíos o duplicidad en el SKU.	UC5

9. Clases de Análisis

Tipo de clase	Propósito en análisis	Ejemplo para CRUD Productos	Sintaxis PlantUML sugerida
Frontera / Boundary	Representan las pantallas limpias y minimalistas que verá el dueño de la papelería. Su única función es mostrar información y capturar los toques en pantalla.	UI_Inventario, UI_FormularioProducto	boundary "Pantalla Gestión de Productos " as UI
Control / Control	Es el cerebro de este módulo. Contiene exactamente los métodos correspondientes a tus requerimientos (Registro Manual [RQF-007], Consulta [RQD-011], Edición [RQF-013] y Eliminación [RQF-023]).	ControladorInventario	control "ControladorInventario" as CtrlInv

Entidad / Entity	La representación de la mercancía. Tiene los atributos esenciales que a un pequeño comerciante le interesa controlar.	Producto	entity "Producto" as Prd
------------------	---	----------	--------------------------



```
@startuml
```

```
skinparam classAttributeIconSize 0
```

```
skinparam shadowing false
```

```
left to right direction
```

```
class "UI_GestionProducto" <<boundary>> {
    - txtSKU: String
    - txtNombre: String
    - txtPrecio: Double
    - txtStock: int
    + clickRegistrarManual()
    + clickRegistrarEscaneo()
    + clickActualizar()
    + clickEliminar()
    + clickConsultarInventario()
    + mostrarMensaje(mensaje: String)
    + mostrarTabla(productos: List<Producto>)
}
```

```
class "ControlInventario" <<control>> {
    + registrarProductoManual(sku: String, nombre: String, precio: Double, stock: int):
boolean
    + registrarProductoEscaneo(codigoBarras: String): boolean
    + actualizarProducto(id: String, sku: String, nombre: String, precio: Double, stock:
int): boolean
    + eliminarProducto(id: String): boolean
    + consultarInventario(): List<Producto>
    + validarDatos(sku: String, nombre: String, precio: Double, stock: int): boolean
}
```

```
class "Producto" <<entity>> {
    - idProducto: String
    - sku: String
    - nombre: String
    - precioUnitario: Double
}
```

```

- stock: int
+ Producto(id: String, sku: String, nombre: String, precio: Double, stock: int)
+ getIdProducto(): String
+ getSku(): String
+ getNombre(): String
+ getPrecioUnitario(): Double
+ getStock(): int
}

class "RepositorioProducto" <<repository>> {
+ existeSKU(sku: String): boolean
+ guardar(producto: Producto): void
+ buscarPorId(id: String): Producto
+ actualizar(producto: Producto): void
+ eliminar(id: String): void
+ listarTodos(): List<Producto>
+ sincronizarNube(): void
}

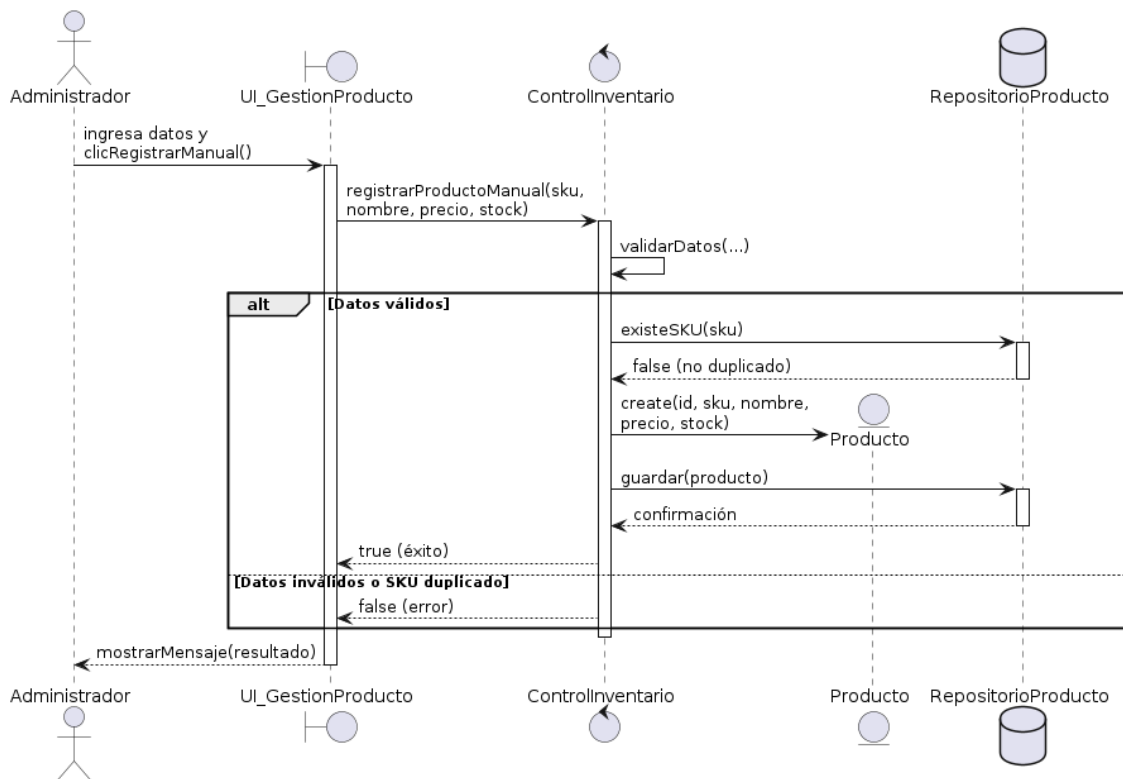
UI_GestionProducto --> ControlInventario : solicita operación
ControlInventario --> Producto : crea/actualiza entidad
ControlInventario --> RepositorioProducto : gestiona persistencia
RepositorioProducto o-- "0..*" Producto : almacena
@enduml

```

9. Secuencias de Análisis

9.1 Secuencia Registro de Producto Manual (Crear / RQF-007)

Es el flujo para cuando el administrador ingresa un producto al inventario. El ControlInventario se encarga de dos validación: verificar los campos y comprobar en el RepositorioProducto que el código SKU no este duplicado.



Código

```

@startuml
skinparam maxMessageSize 150
actor Administrador as Admin
boundary "UI_GestionProducto" as UI
control "ControllInventario" as Ctrl
entity "Producto" as Prod
database "RepositorioProducto" as Repo

Admin -> UI : ingresa datos y clicRegistrarManual()
activate UI

UI -> Ctrl : registrarProductoManual(sku, nombre, precio, stock)
activate Ctrl

Ctrl -> Ctrl : validarDatos(...)
alt Datos válidos
    Ctrl -> Repo : existeSKU(sku)
    activate Repo
    Repo --> Ctrl : false (no duplicado)
    deactivate Repo
    Ctrl -> Prod : create(id, sku, nombre, precio, stock)
    activate Prod
    deactivate Prod
    Ctrl -> Repo : guardar(producto)
    activate Repo
    Repo --> Ctrl : confirmación
    deactivate Repo
    Ctrl --> UI : true (éxito)
else Datos inválidos o SKU duplicado
    Ctrl --> UI : false (error)
end
deactivate Ctrl

UI -> Admin : mostrarMensaje(resultado)
deactivate UI
    
```

```

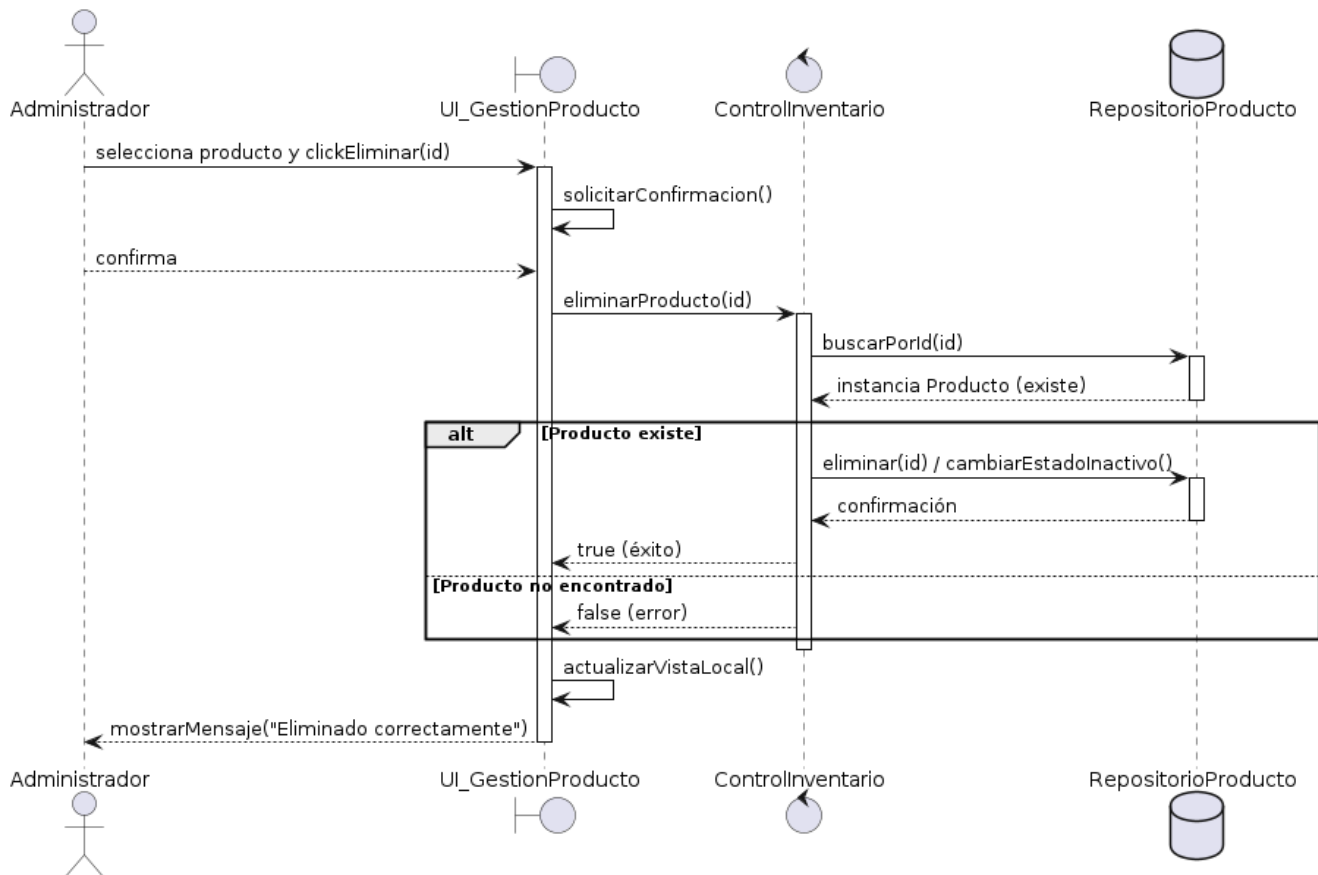
end
deactivate Ctrl

UI --> Admin : mostrarMensaje(resultado)
deactivate UI
@enduml

```

9.2 Secuencia Eliminar / Dar de baja Producto (Borrar / RQF-023)

El controlador busca ID, valida que este exista y ejecuta la orden al repositorio. Previo a esto se requiere la confirmación explícita del administrador.



Código

```

@startuml
actor Administrador as Admin
boundary "UI_GestionProducto" as UI
control "ControllInventario" as Ctrl
database "RepositorioProducto" as Repo

Admin -> UI : selecciona producto y clickEliminar(id)
activate UI

UI -> UI : solicitarConfirmacion()
Admin --> UI : confirma
deactivate Admin

UI -> Ctrl : eliminarProducto(id)
activate Ctrl
Ctrl -> Repo : buscarPorId(id)
activate Repo
Repo --> Ctrl : instancia Producto (existe)
deactivate Repo
alt [Producto existe]
    Ctrl -> Repo : eliminar(id) / cambiarEstadoInactivo()
    activate Repo
    Repo --> Ctrl : confirmación
    deactivate Repo
else [Producto no encontrado]
    Ctrl --> UI : false (error)
end
Ctrl --> UI : true (éxito)
deactivate Ctrl
UI -> UI : actualizarVistaLocal()
UI --> Admin : mostrarMensaje("Eliminado correctamente")
deactivate UI

```

```

UI -> Ctrl : eliminarProducto(id)
activate Ctrl

Ctrl -> Repo : buscarPorId(id)
activate Repo
Repo --> Ctrl : instancia Producto (existe)
deactivate Repo

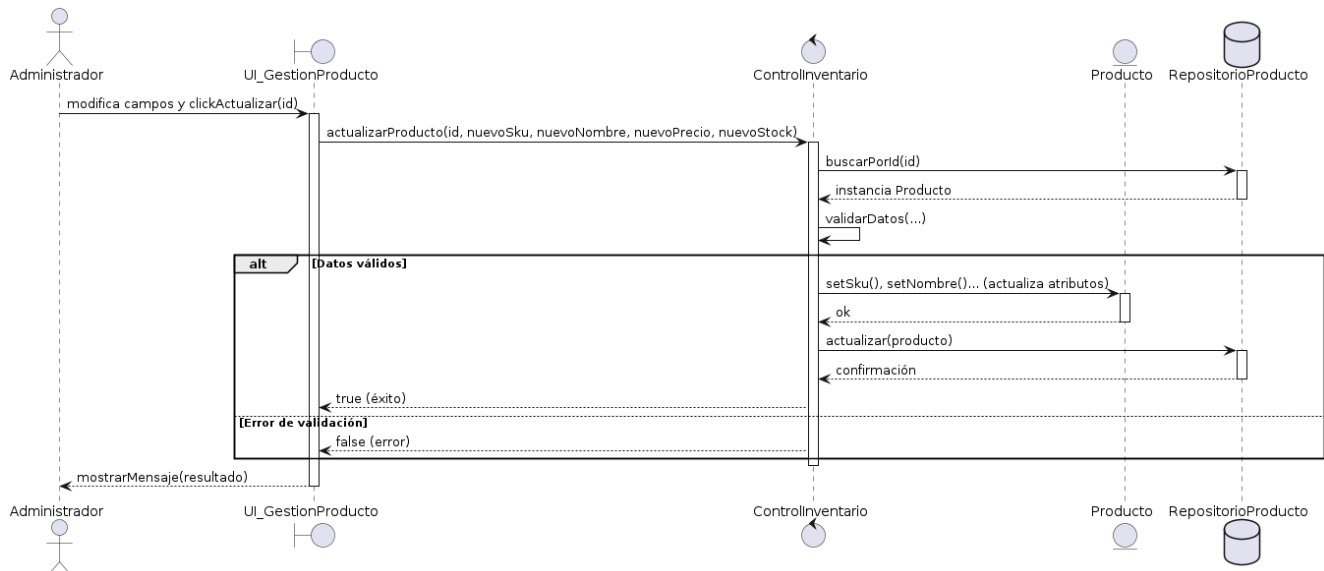
alt Producto existe
  Ctrl -> Repo : eliminar(id) / cambiarEstadoInactivo()
  activate Repo
  Repo --> Ctrl : confirmación
  deactivate Repo
  Ctrl --> UI : true (éxito)
else Producto no encontrado
  Ctrl --> UI : false (error)
end
deactivate Ctrl

UI -> UI : actualizarVistaLocal()
UI --> Admin : mostrarMensaje("Eliminado correctamente")
deactivate UI
@enduml

```

9.3 Secuencia Editar Productos (Actualizar / RQF-013)

Una vez identificado un producto, el usuario puede editar los campos de la interfaz, el controlador recupera el producto original desde el repositorio, valida los datos nuevo, actualiza atributos y ordena el repositorio.



Código

```

@startuml
actor Administrador as Admin
boundary "UI_GestionProducto" as UI

```

```

control "ControllInventario" as Ctrl
entity "Producto" as Prod
database "RepositorioProducto" as Repo

Admin -> UI : modifica campos y clickActualizar(id)
activate UI

UI -> Ctrl : actualizarProducto(id, nuevoSku, nuevoNombre, nuevoPrecio, nuevoStock)
activate Ctrl

Ctrl -> Repo : buscarPorId(id)
activate Repo
Repo --> Ctrl : instancia Producto
deactivate Repo

Ctrl -> Ctrl : validarDatos(...)
alt Datos válidos
  Ctrl -> Prod : setSku(), setNombre()... (actualiza atributos)
  activate Prod
  Prod --> Ctrl : ok
  deactivate Prod

  Ctrl -> Repo : actualizar(producto)
  activate Repo
  Repo --> Ctrl : confirmación
  deactivate Repo

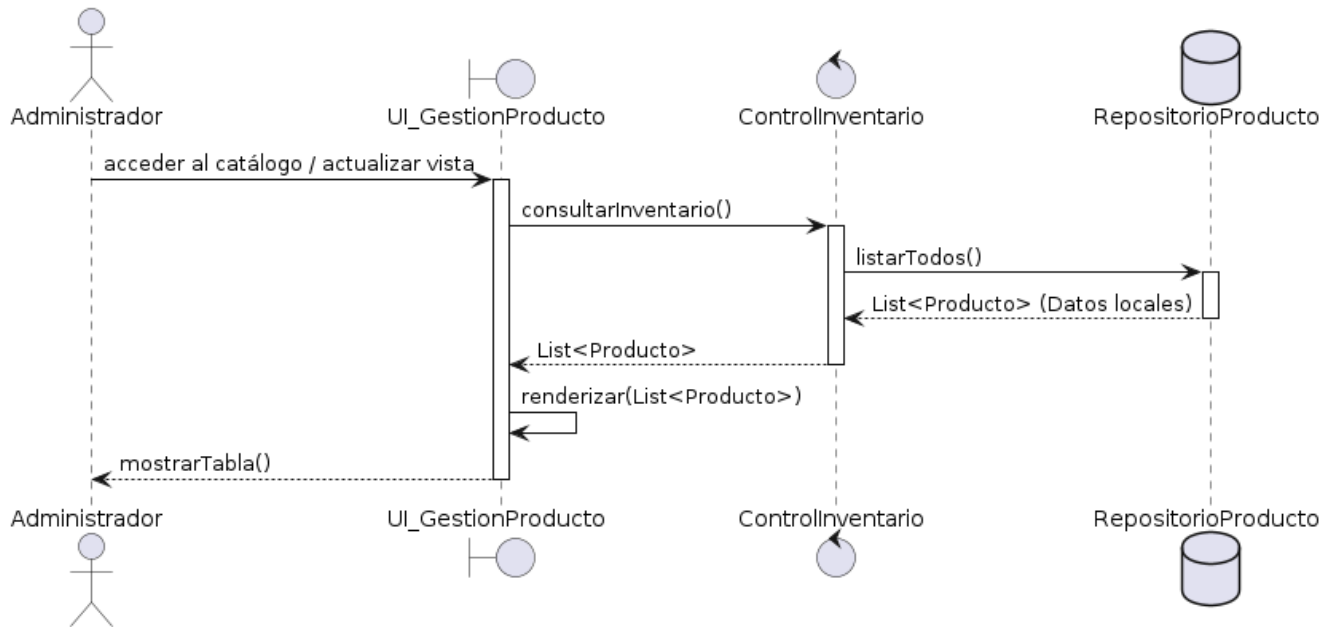
  Ctrl --> UI : true (éxito)
else Error de validación
  Ctrl --> UI : false (error)
end
deactivate Ctrl

UI --> Admin : mostrarMensaje(resultado)
deactivate UI
@enduml

```

9.4 Secuencia Consulta de Productos en Inventario (Leer / RQF-011)

Es la operación más rápida e intuitiva. Se activa cuando el usuario entra en la pantalla principal, la interfaz solicita los datos al controlador, este extrae los datos del repositorio y lista los objetos Producto en la UI.



Código

```

@startuml
actor Administrador as Admin
boundary "UI_GestionProducto" as UI
control "ControlInventario" as Ctrl
database "RepositorioProducto" as Repo

Admin -> UI : acceder al catálogo / actualizar vista
activate UI

UI -> Ctrl : consultarInventario()
activate Ctrl

Ctrl -> Repo : listarTodos()
activate Repo
Repo --> Ctrl : List<Producto> (Datos locales)
deactivate Repo

Ctrl --> UI : List<Producto>
deactivate Ctrl

UI -> UI : renderizar(List<Producto>)
UI --> Admin : mostrarTabla()
deactivate UI
@enduml
  
```

10. Casos de Uso

A continuación, se define el Caso de Uso (CU) correspondiente a cada Requisito Funcional (RF) establecido, detallando su propósito y relación para sentar las bases del modelado UML.

1. Requisito Funcional 007 (RQF-007)

"El sistema debe permitir registrar un nuevo producto ingresando manualmente sus datos (Nombre, SKU, Precio, Stock, etc.) en caso de no poseer o no utilizar código de barras."

Caso de Uso	CU-04: Registrar Producto Manualmente
Actor Principal	Administrador
Descripción	Permite al administrador ingresar los datos de un nuevo producto mediante un formulario para añadirlo al inventario.
Relaciones	<<include>> CU-Validar: Validar unicidad de SKU y campos obligatorios (RQF-008).

Código PlantUML (Diagrama Individual CU-04)

```

@startuml
left to right direction
skinparam packageStyle rectangle

actor "Administrador" as Admin

rectangle "Sistema PaperBoost" {
    usecase "Registrar Producto Manualmente" as UC1
    usecase "Validar Unicidad de SKU" as UC2
}

Admin -- UC1
UC1 ..> UC2 : <<include>>

@enduml

```



2. Requisito Funcional 011 (RQF-011)

"El sistema debe permitir consultar el inventario de productos mostrando la lista completa y resaltando aquellos con stock crítico."

Caso de Uso	CU-06: Consultar Inventario de Productos
Actor Principal	Administrador
Descripción	Permite al administrador visualizar el listado de todos los productos registrados en el sistema de manera rápida.
Relaciones	<<extend>> CU-Filtros: Aplicar filtros de búsqueda y ordenamiento (RQF-012).

Código PlantUML (Diagrama Individual CU-06)

```

@startuml
left to right direction
skinparam packageStyle rectangle

actor "Administrador" as Admin

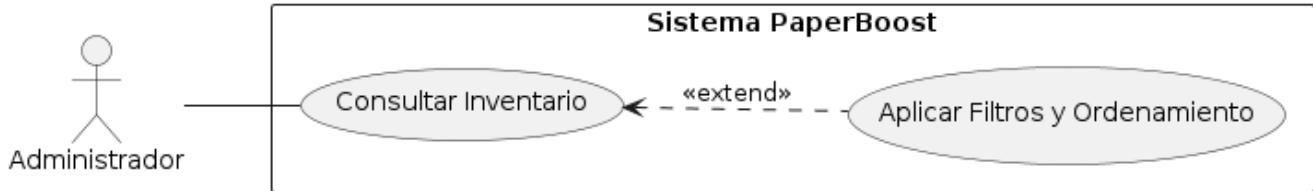
rectangle "Sistema PaperBoost" {
    usecase "Consultar Inventario" as UC1
    usecase "Aplicar Filtros y Ordenamiento" as UC2
}

```

Admin -- UC1

UC1 <.. UC2 : <<extend>>

@enduml



3. Requisito Funcional 013 (RQF-013)

"El sistema debe permitir la edición o modificación de los datos de un producto previamente registrado en el catálogo."

Caso de Uso	CU-07: Modificar datos de producto existente
Actor Principal	Administrador
Descripción	Permite al usuario actualizar la información de un producto (como cambiar su precio o actualizar el nombre) seleccionándolo de la lista.
Relaciones	<<include>> CU-Validar: Validar los nuevos datos ingresados antes de guardar.

Código PlantUML (Diagrama Individual CU-07)

@startuml

left to right direction

skinparam packageStyle rectangle

```

actor "Administrador" as Admin

rectangle "Sistema PaperBoost" {
    usecase "Modificar Datos de Producto" as UC1
    usecase "Validar Nuevos Datos" as UC2
}

Admin -- UC1
UC1 ..> UC2 : <<include>>
@enduml

```



4. Requisito Funcional 023 (RQF-023)

"El sistema debe permitir eliminar o dar de baja un producto del inventario para que ya no esté disponible en futuras transacciones."

Caso de Uso	CU-12: Eliminar / Dar de baja producto
Actor Principal	Administrador
Descripción	Permite al administrador remover un producto del catálogo activo seleccionándolo y confirmando la acción.
Relaciones	<<include>> CU-Confirmar: Solicitar confirmación obligatoria de eliminación (RQF-024).

Código PlantUML (Diagrama Individual CU-12)

```

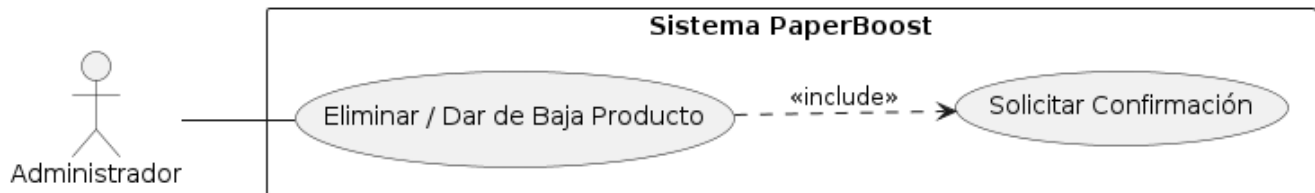
@startuml
left to right direction
skinparam packageStyle rectangle

actor "Administrador" as Admin

rectangle "Sistema PaperBoost" {
    usecase "Eliminar / Dar de Baja Producto" as UC1
    usecase "Solicitar Confirmación" as UC2
}

Admin -- UC1
UC1 ..> UC2 : <<include>>
@enduml

```



10. Matriz de trazabilidad RF-CU

Código RF	Requisito Funcional	Caso de Uso Asociado	Clases de Análisis Involucradas	Diagrama de Secuencia
RQF-007	Permitir registrar un nuevo producto manualmente.	Registrar Producto Manualmente (CUC-04)	UI_GestionProducto, ControlInventario, Producto, RepositorioProducto	Secuencia: Registro de Producto Manual

RQF-013	Permitir la edición o modificación de un producto.	Modificar datos de producto (CUC-07)	UI_GestionProducto, ControlInventario, Producto, RepositorioProducto	Secuencia: Editar Productos
RQF-023	Permitir eliminar o dar de baja un producto del inventario.	Eliminar / Dar de baja producto (CUC-12)	UI_GestionProducto, ControlInventario, RepositorioProducto	Secuencia: Eliminar / Dar de baja Producto
RQF-011	Permitir consultar el inventario de productos.	Consultar Inventario de Productos (CUC-06)	UI_GestionProducto, ControlInventario, RepositorioProducto	Secuencia: Consulta de Productos en Inventario
RQF-008	Validar campos obligatorios y unicidad.	Validar campos y SKU (Include)	ControlInventario	Integrado en secuencias de Registro y Edición

11. Lista de verificación para revisar el diagrama

- Cada caso de uso se deriva de un requisito funcional de nivel 0.
- El actor representa un rol externo, no una pantalla ni una clase interna.
- El nombre del caso de uso está redactado como objetivo del usuario.
- El límite del sistema está claramente identificado.
- Las relaciones <<include>> o <<extend>> tienen justificación funcional.
- El diagrama no modela detalles técnicos prematuros.
- El código PlantUML compila y genera un diagrama legible.
- La matriz RF-CU permite verificar trazabilidad sin pérdida de información.

12. Conclusiones técnicas

- El análisis evidencia un flujo estructurado donde los requisitos funcionales de nivel 0 actúan como el origen formal del sistema, definiendo el *qué* sin introducir decisiones de diseño. Cada requisito se transforma directamente en un caso de uso, garantizando una correspondencia uno a uno que evita ambigüedad y asegura cobertura funcional completa. Este mapeo se consolida mediante relaciones como <<include>>, especialmente en el caso de validación, lo que demuestra la identificación de comportamientos transversales reutilizables. El paso crítico ocurre en el “puente” hacia las clases de análisis, donde los casos de uso dejan de ser descripciones y se convierten en responsabilidades distribuidas: la frontera gestiona interacción, el control centraliza la lógica del caso de uso, y la entidad representa el estado del dominio. Este punto marca la transición de análisis funcional a diseño orientado a objetos, estableciendo una base clara para una arquitectura desacoplada y coherente.
- El modelo demuestra una trazabilidad entre todos los artefactos, donde cada requisito funcional se refleja consistentemente en un caso de uso, en métodos concretos dentro del controlador y en flujos explícitos dentro de los diagramas de secuencia. Esta alineación permite validar que la estructura estática (clases y métodos) responde exactamente al comportamiento dinámico esperado (interacciones en secuencia), evitando inconsistencias entre lo que el sistema debe hacer y cómo se implementa. Las secuencias cumplen un rol clave al verificar la correcta asignación de responsabilidades, evidenciando que la lógica reside en el controlador, la persistencia en el repositorio y la entidad se mantiene como un modelo de datos sin control de flujo. La matriz de trazabilidad

consolida esta relación, permitiendo auditar el sistema, evaluar impacto de cambios y garantizar que no existan funcionalidades sin respaldo en requisitos.

